

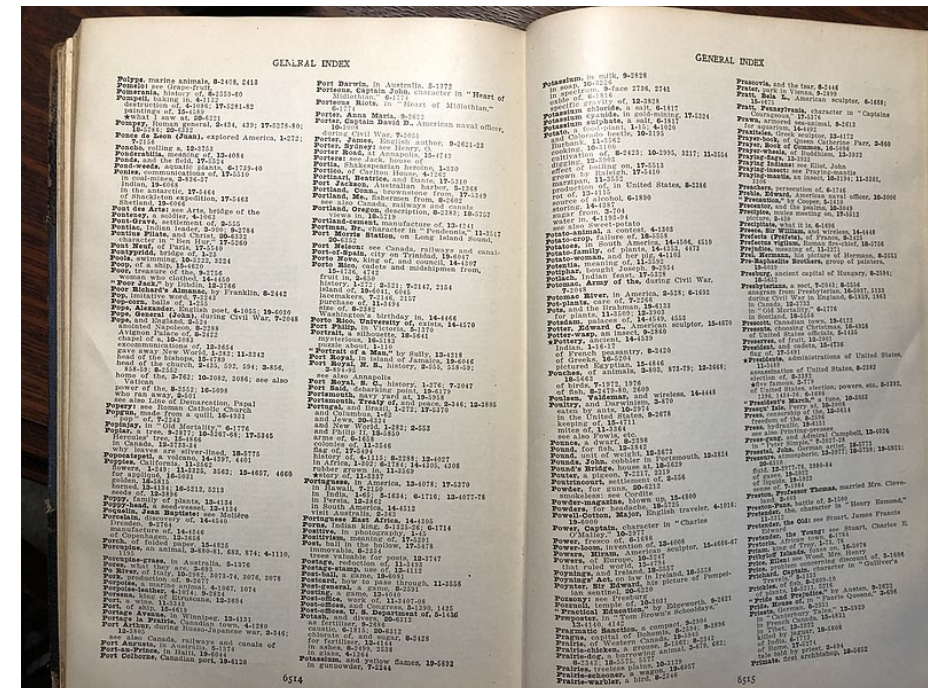
Inverted index. Boolean retrieval

Pavel Braslavski

slides borrowed from
[Stanford's CS 276 / LING 286](#)
[Speech and language processing, 3rd edition](#)

Pre-history

- First indices ~3000 B.C.
- Libraries and books
- Computer-powered search ~1950



Term-document matrix

Unstructured data in 1620

- Which plays of Shakespeare contain the words ***Brutus AND Caesar*** but ***NOT Calpurnia***?
- One could `grep` all of Shakespeare's plays for ***Brutus*** and ***Caesar***, then strip out lines containing ***Calpurnia***?
- Why is that not the answer?
 - Slow (for large corpora)
 - ***NOT Calpurnia*** is non-trivial
 - Other operations (e.g., find the word ***Romans*** near ***countrymen***) not feasible
 - Ranked retrieval (best documents to return)
 - Later lectures

Term-document incidence matrices

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

***Brutus AND Caesar BUT NOT
Calpurnia***

1 if play contains
word, 0 otherwise

Incidence vectors

- So we have a 0/1 vector for each term.
- To answer query: take the vectors for ***Brutus***, ***Caesar*** and ***Calpurnia*** (complemented) → bitwise *AND*.
 - 110100 *AND*
 - 110111 *AND*
 - 101111 =
 - **100100**

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0

Answers to query

- Antony and Cleopatra, Act III, Scene ii

Agrippa [Aside to DOMITIUS ENOBARBUS]: Why, Enobarbus,

When Antony found Julius **Caesar** dead,
He cried almost to roaring; and he wept
When at Philippi he found **Brutus** slain.

- Hamlet, Act III, Scene ii

Lord Polonius: I did enact Julius **Caesar** I was killed i' the
Capitol; **Brutus** killed me.

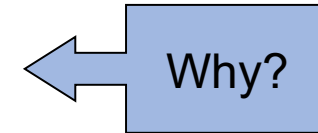


Bigger collections

- Consider $N = 1$ million documents, each with about 1000 words.
- Avg 6 bytes/word including spaces/punctuation
 - 6GB of data in the documents.
- Say there are $M = 500K$ *distinct* terms among these.

Can't build the matrix

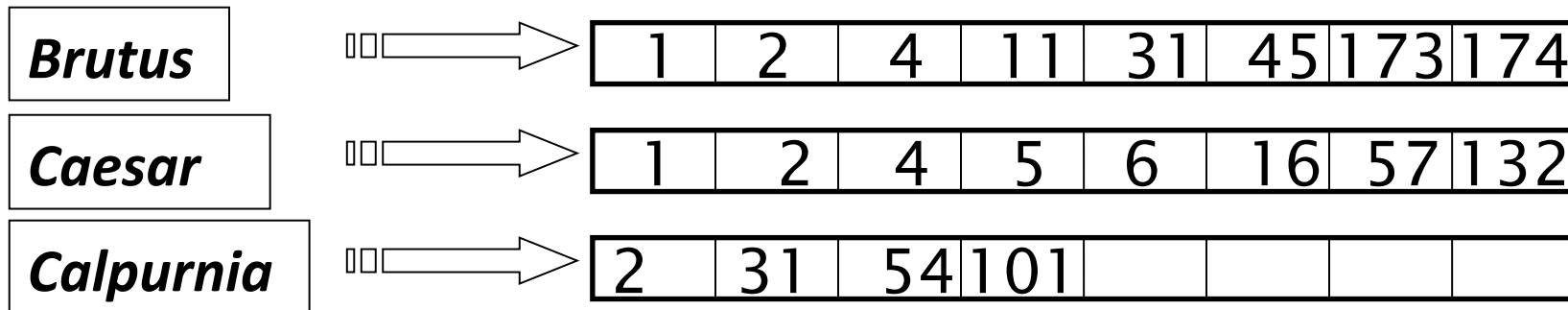
- 500K x 1M matrix has half-a-trillion 0's and 1's.
- But it has no more than one billion 1's.
 - matrix is extremely sparse.
- What's a better representation?
 - We only record the 1 positions.



Inverted index

Inverted index

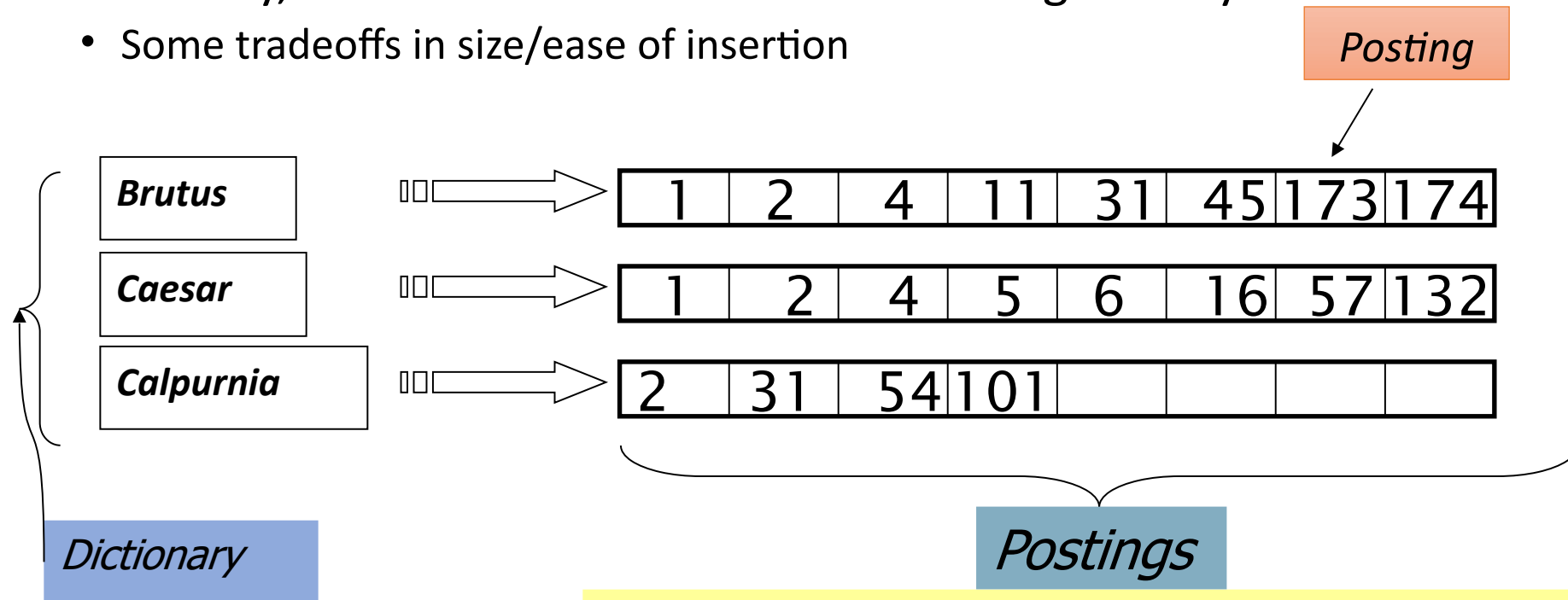
- For each term t , we must store a list of all documents that contain t .
 - Identify each doc by a **docID**, a document serial number
- Can we use fixed-size arrays for this?



What happens if the word **Caesar** is added to document 14?

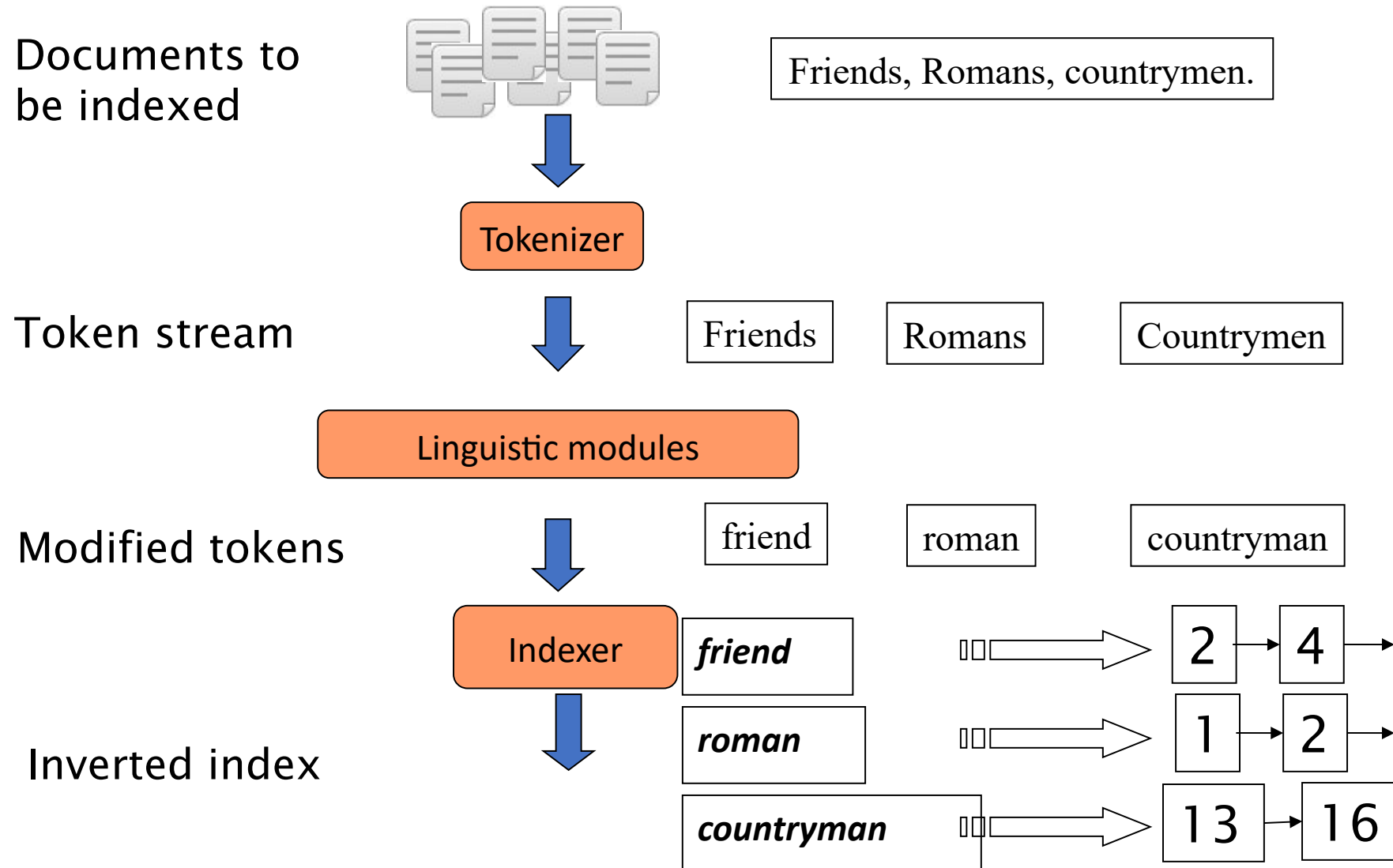
Inverted index

- We need variable-size **postings lists**
 - On disk, a continuous run of postings is normal and best
 - In memory, can use linked lists or variable length arrays
 - Some tradeoffs in size/ease of insertion



Sorted by docID (more later on why).

Inverted index construction



Document processing

Initial stages of text processing

- Tokenization
 - Cut character sequence into word tokens
 - Deal with ***“John’s”, a state-of-the-art solution***
- Normalization
 - Map text and query term to same form
 - You want **U.S.A.** and **USA** to match
- Stemming
 - We may wish different forms of a root to match
 - ***authorize, authorization***
- Stop words
 - We may omit very common words (or not)
 - ***the, a, to, of***

Space-based tokenization

- A very simple way to tokenize
 - For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
 - Segment off a token between instances of spaces + punctuation

Issues in Tokenization

- Can't just blindly remove punctuation:
 - [m.p.h.](#), [Ph.D.](#), [AT&T](#), [cap'n](#)
 - prices ([\\$45.55](#))
 - dates ([01/02/06](#))
 - URLs (<http://www.stanford.edu>)
 - hashtags ([#nlproc](#))
 - email addresses (someone@cs.colorado.edu)
- Clitic: a word that doesn't stand on its own
 - "are" in [we're](#), French "je" in [j'ai](#), "le" in [l'honneur](#), German "in dem" in "im"
- When should multiword expressions (MWE) be words?
 - [New York](#), [rock 'n' roll](#)

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...     ([A-Z]\.)+          # abbreviations, e.g. U.S.A.
...     | \w+(-\w+)*        # words with optional internal hyphens
...     | \$?\d+(\.\d+)?%?   # currency and percentages, e.g. $12.40, 82%
...     | \.\.\.            # ellipsis
...     | [][.,;"'()?:_']  # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization in different languages

- DE: Sauerstoffverbrauchsrate, Lebensversicherungsreformgesetz
- TR: Bayramlasamadiklarimiz –
Those with whom we cannot exchange festival greetings
[because they have died in the intervening period since last Bayram]
- ZH:

S	S	B	E	B	M	E	S
我	有	一	台	计	算	机	。
(I)	(have)	(a)		(computer)			(.)

Tokenization in languages without spaces

Many languages (like Chinese, Japanese, Thai) don't use spaces to separate words!

How do we decide where the token boundaries should be?

Word tokenization in Chinese

Chinese words are composed of characters called "**hanzi**" (or sometimes just "**zi**")

Each one represents a meaning unit called a morpheme.

Each word has on average 2.4 of them.

But deciding what counts as a word is complex and not agreed upon.

How to do word tokenization in Chinese?

- 姚明进入总决赛 “ Yao Ming reaches the finals”

3 words?

- 姚明 进入 总决赛
- YaoMing reaches finals

5 words?

- 姚 明 进入 总 决赛
- Yao Ming reaches overall finals

7 characters? (don't use words at all):

- 姚 明 进 入 总 决 赛
- Yao Ming enter enter overall decision game

Word tokenization / segmentation

So in Chinese it's common to just treat each character (zi) as a token.

- So the **segmentation** step is very simple

In other languages (like Thai and Japanese), more complex word segmentation is required.

- The standard algorithms are neural sequence models trained by supervised machine learning.

Token normalization

Porter Stemmer

- Based on a series of rewrite rules run in series
 - A cascade, in which output of each pass fed to next pass
- Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ε if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

Porter stemmer

It is a truth universally acknowledged that a single man in possession of a good fortune must be in want of a wife. However little known the feelings or views of such a man may be on his first entering a neighbourhood, this truth is so well fixed in the minds of the surrounding families that he is considered the rightful property of some one or other of their daughters.

Russian stemmer

Step 1: Search for a PERFECTIVE GERUND ending. If one is found remove it, and that is then the end of step 1. Otherwise try and remove a REFLEXIVE ending, and then search in turn for (1) an ADJECTIVAL, (2) a VERB or (3) a NOUN ending. As soon as one of the endings (1) to (3) is found remove it, and terminate step 1.

Step 2: If the word ends with и (*i*), remove it.

Step 3: Search for a DERIVATIONAL ending in *R2* (i.e. the entire ending must lie in *R2*), and if one is found, remove it.

Step 4: (1) Undouble н (*n*), or, (2) if the word ends with a SUPERLATIVE ending, remove it and undouble н (*n*), or (3) if the word ends ь (') (soft sign) remove it.

<http://snowball.tartarus.org/algorithms/russian/stemmer.html>

...doesn't work well actually

кровать – крoвa

кроватью – крoвaт

лью – лью

лить – лит

людей – люд

человека – человек

лучше – лучш

хорошеет – хорошеет

бежит – беж

бегу – бег

Lemmatization

Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- **Uygarlastiramadiklarimizdanmissinizcasina**
- `(behaving) as if you are among those whom we could not civilize’
- **Uygar** `civilized’ + **las** `become’
 - + **tir** `cause’ + **ama** `not able’
 - + **dik** `past’ + **lar** `plural’
 - + **imiz** `p1pl’ + **dan** `abl’
 - + **mis** `past’ + **siniz** `2pl’ + **casina** `as if’

Lemmatization – example for Russian

В больничном дворе стоит небольшой флигель, окруженный целым лесом репейника, крапивы и дикой конопли.

в больничный двор стоять небольшой флигель окружать целый лес
репейник крапива и дикий конопля

Russian nouns and adjectives

падеж	ед. ч.	мн. ч.
Им.	ча́й	чаи́
Р.	ча́я	чаёв
Д.	ча́ю	чая́м
В.	ча́й	чаи́
Тв.	ча́ем	чая́ми
Пр.	ча́е	чая́х
Разд.	ча́ю	—

падеж	ед. ч.			мн. ч.
	муж. р.	ср. р.	жен. р.	
Им.	красный	красное	красная	красные
Рд.	красного	красного	красной	красных
Дт.	красному	красному	красной	красным
Вн.	одуш. красного	красное	красную	красных
	неод. красный			красные
Тв.	красным	красным	красной красною	красными
Пр.	красном	красном	красной	красных
Кратк. форма	красен	красно красно́	красна́	красны красны́

Russian verbs

	наст.	прош.	повелит.
Я	пью	пил пила	—
Ты	пьёшь	пил пила	пей
Он Она Оно	пьёт	пил пила пило	—
Мы	пьём	пили	—
Вы	пьёте	пили	пейте
Они	пьют	пили	—
Пр. действ. наст.	пьющий		
Пр. действ. прош.	пивший		
Деепр. наст.	—		
Деепр. прош.	пив, пивши		
Пр. страд. наст.	пityй		
Будущее	буду/будешь... пить		

падеж	ед. ч.			мн. ч.
	муж. р.	ср. р.	жен. р.	
Им.	пьющий	пьющее	пьющая	пьющие
Рд.	пьющего	пьющего	пьющей	пьющих
Дт.	пьющему	пьющему	пьющей	пьющим
Вн.	одуш.	пьющего	пьющее пьющую	пьющих
	неод.	пьющий		пьющие
Тв.	пьющим	пьющим	пьющей пьющею	пьющими
Пр.	пьющем	пьющем	пьющей	пьющих

падеж	ед. ч.			мн. ч.
	муж. р.	ср. р.	жен. р.	
Им.	пивший	пившее	пившая	пившие
Рд.	пившего	пившего	пившей	пивших
Дт.	пившему	пившему	пившей	пившим
Вн.	одуш.	пившего	пившее пившую	пивших
	неод.	пивший		пившие
Тв.	пившим	пившим	пившей пившею	пившими
Пр.	пившем	пившем	пившей	пивших

Lemmatizers: data/methods

- Manual rules
 - Word form – lemma pairs
 - Annotated text (context!)
-
- Finite state transducers
 - Dictionaries
 - Neural networks

Data source

Зализняк А.А. Грамматический словарь русского языка (1977)

~100 тыс. входов

Модель русского словоизменения

Пример: лев мо 1*b (животное)

лев м 1а (денежная единица)

стричь нсв 8b (-г-)

приходя ж (п 4а)

Morphology & search quality

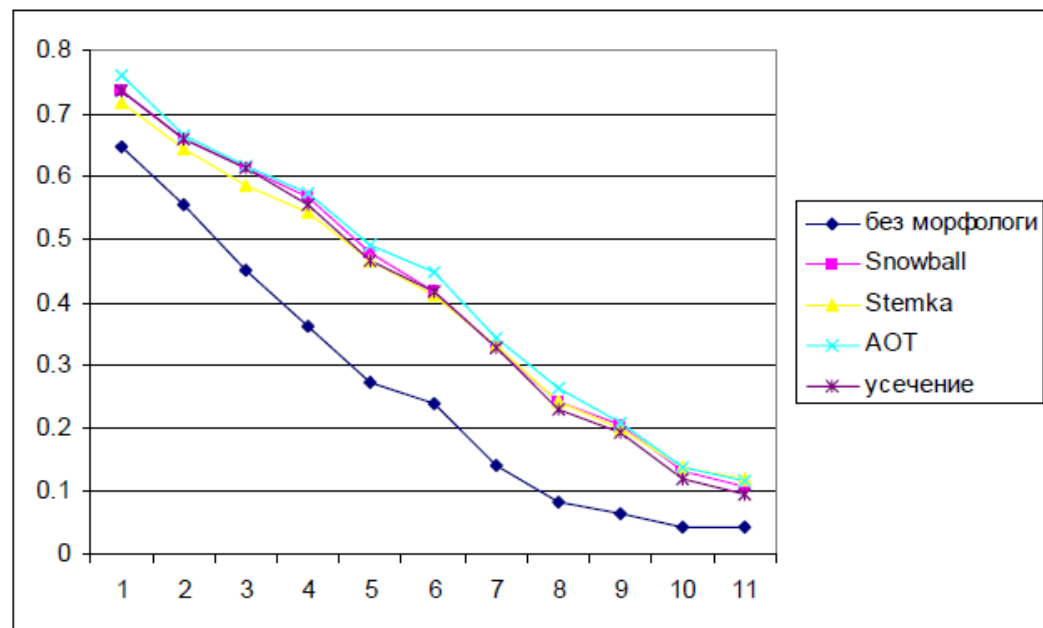


Рисунок 1. Коллекция legal, строгая релевантность

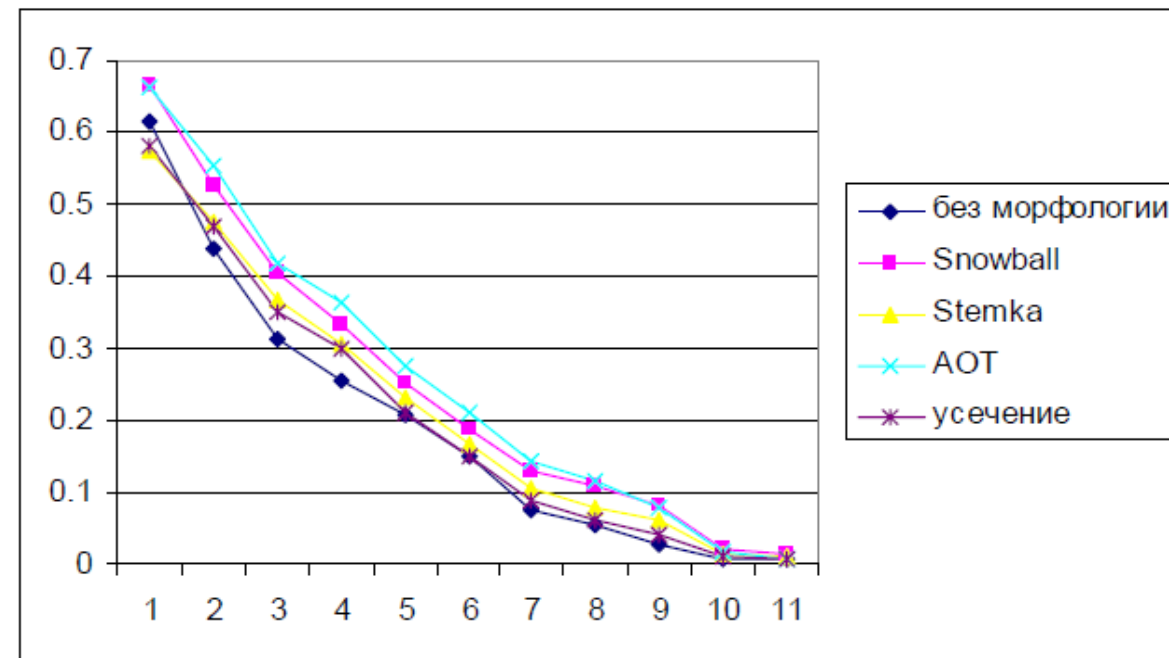


Рисунок 2. Коллекция Web, слабая релевантность

Губин М., Морозов А. Влияние морфологического анализа на качество информационного поиска. RCDL'2006.

Subword tokenization

Another option for text tokenization

Instead of

- white-space segmentation
- single-character segmentation

Use the data to tell us how to tokenize.

Subword tokenization (because tokens can be parts of words as well as whole words)

Subword tokenization

Three common algorithms:

- **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
- **Unigram language modeling tokenization** (Kudo, 2018)
- **WordPiece** (Schuster and Nakajima, 2012)

All have 2 parts:

- A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
- A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,..., a, b, c, d....}

Repeat:

- Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
- Add a new merged symbol 'AB' to the vocabulary
- Replace every adjacent 'A' 'B' in the corpus with 'AB'.

Until k merges have been done.

Byte Pair Encoding (BPE) Addendum

Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '___' before space in training corpus

Next, separate into letters.

BPE token learner

Original (very fascinating 🤪) corpus:

low low low low low lowest lowest newer newer newer newer newer
newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

vocabulary

—, d, e, i, l, n, o, r, s, t, w

BPE token learner

corpus

```
5   l o w _  
2   l o w e s t _  
6   n e w e r _  
3   w i d e r _  
2   n e w _
```

vocabulary

```
_, d, e, i, l, n, o, r, s, t, w
```

Merge **e r** to **er**

corpus

```
5   l o w _  
2   l o w e s t _  
6   n e w e r _  
3   w i d e r _  
2   n e w _
```

vocabulary

```
_, d, e, i, l, n, o, r, s, t, w, er
```

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, e r

Merge **er _** to **er_**

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, e r, e r

BPE

corpus

5 l o w _
2 l o w e s t _
6 n e w er_
3 w i d er_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

Merge n e to ne

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

BPE

The next merges are:

Merge	Current Vocabulary
(ne, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new
(l, o)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo
(lo, w)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low
(new, er—)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—
(low, —)	—, d, e, i, l, n, o, r, s, t, w, er, er—, ne, new, lo, low, newer—, low—

BPE token **segmenter** algorithm

On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them
- (test frequencies don't play a role)

So: merge every **e r** to **er**, then merge **er _** to **er_**, etc.

Result:

- Test set "n e w e r _" would be tokenized as a full word
- Test set "l o w e r _" would be two tokens: "low er_"

Properties of BPE tokens

Usually include frequent words

And frequent subwords

- Which are often morphemes like *-est* or *-er*

A **morpheme** is the smallest meaning-bearing unit of a language

- *unlikeliest* has 3 morphemes *un-*, *likeli*, and *-est*



I quite like the new DeepSeek-OCR paper. It's a good OCR model (maybe a bit worse than dots), and yes data collection etc., but anyway it doesn't matter.

The more interesting part for me (esp as a computer vision at heart who is temporarily masquerading as a natural language person) is whether pixels are better inputs to LLMs than text. Whether text tokens are wasteful and just terrible, at the input.

Maybe it makes more sense that all inputs to LLMs should only ever be images. Even if you happen to have pure text input, maybe you'd prefer to render it and then feed that in:

- more information compression (see paper) => shorter context windows, more efficiency
- significantly more general information stream => not just text, but e.g. bold text, colored text, arbitrary images.
- input can now be processed with bidirectional attention easily and as default, not autoregressive attention - a lot more powerful.
- delete the tokenizer (at the input)!! I already ranted about how much I dislike the tokenizer. Tokenizers are ugly, separate, not end-to-end stage. It "imports" all the ugliness of Unicode, byte encodings, it inherits a lot of historical baggage, security/jailbreak risk (e.g. continuation bytes). It makes two characters that look identical to the eye look as two completely different tokens internally in the network. A smiling emoji looks like a weird token, not an... actual smiling face, pixels and all, and all the transfer learning that brings along. The tokenizer must go.

Index construction

Indexer steps: Token sequence

- Sequence of (Modified token, Document ID) pairs.

Doc 1

I did enact Julius
Caesar I was killed
i' the Capitol;
Brutus killed me.

Doc 2

So let it be with
Caesar. The noble
Brutus hath told you
Caesar was ambitious



Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2

Indexer steps: Sort

- Sort by terms
 - At least conceptually
 - And then docID

Core indexing step

Term	docID
I	1
did	1
enact	1
julius	1
caesar	1
I	1
was	1
killed	1
i'	1
the	1
capitol	1
brutus	1
killed	1
me	1
so	2
let	2
it	2
be	2
with	2
caesar	2
the	2
noble	2
brutus	2
hath	2
told	2
you	2
caesar	2
was	2
ambitious	2



Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2

Indexer steps: Dictionary & Postings

- Multiple term entries in a single document are merged.
- Split into Dictionary and Postings
- Doc. frequency information is added.

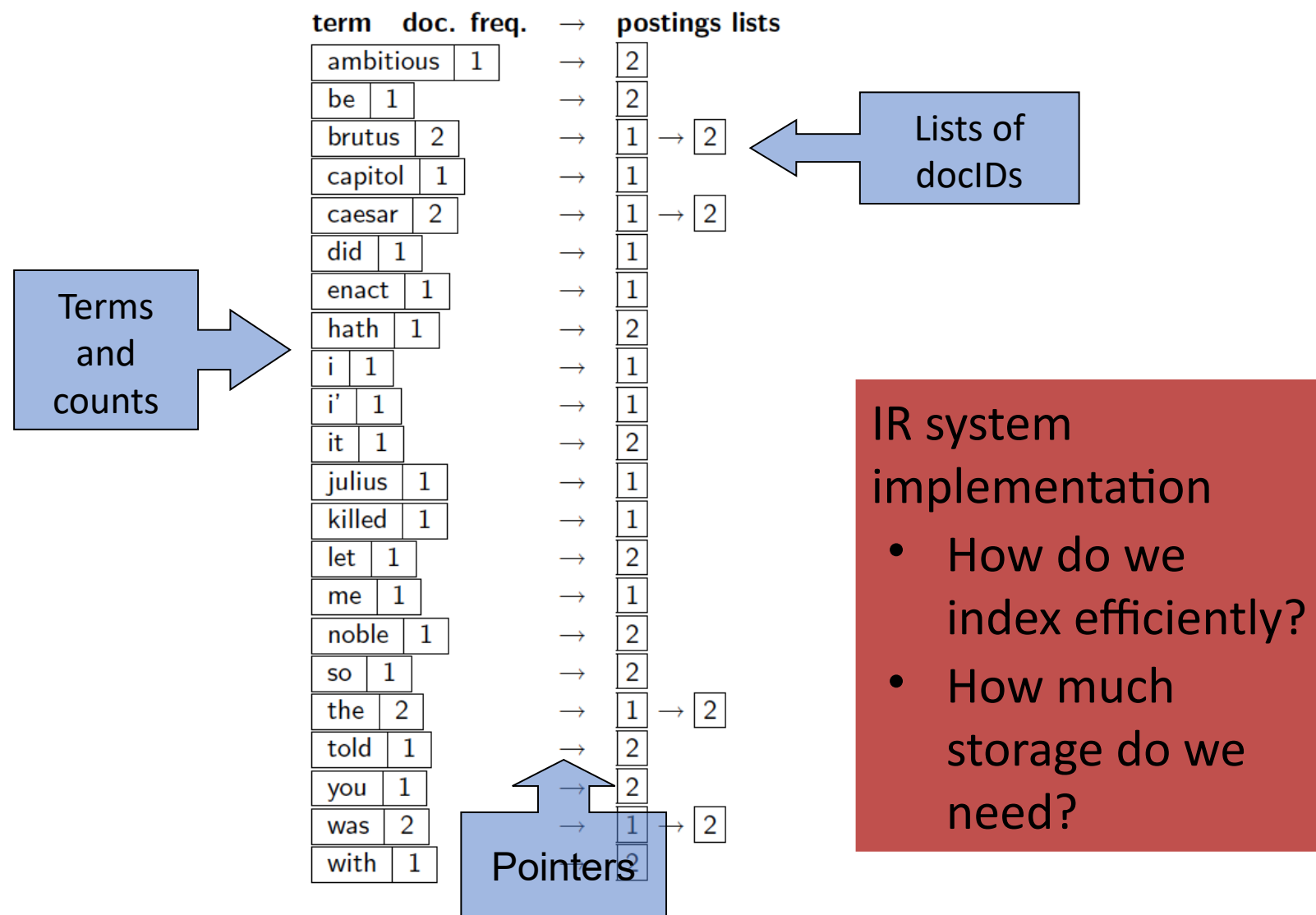
Why frequency?
Will discuss later.

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
I	1
I	1
i'	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	2
the	1
the	2
told	2
you	2
was	1
was	2
with	2



term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
i	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Where do we pay in storage?



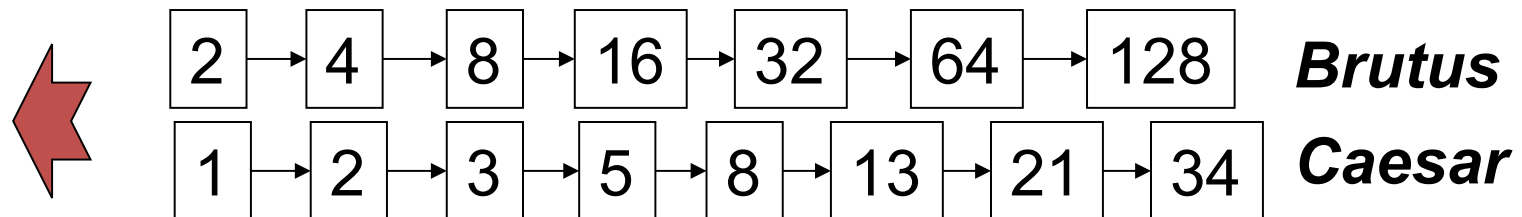
Query processing

Query processing: AND

- Consider processing the query:

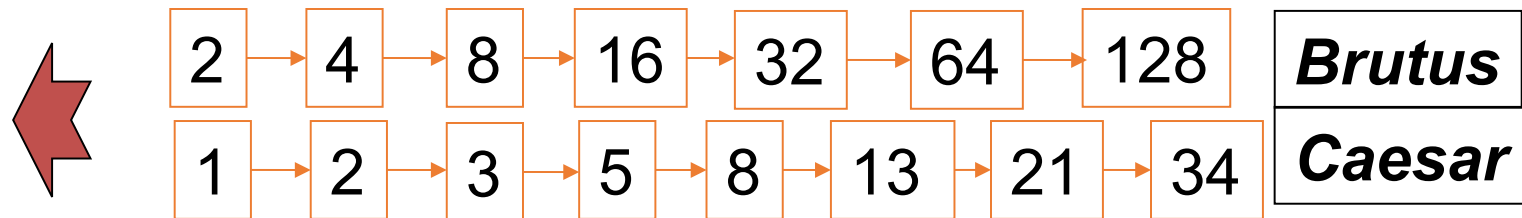
Brutus AND Caesar

- Locate ***Brutus*** in the Dictionary;
 - Retrieve its postings.
- Locate ***Caesar*** in the Dictionary;
 - Retrieve its postings.
- “Merge” the two postings (intersect the document sets):



The merge

- Walk through the two postings simultaneously, in time linear in the total number of postings entries



If the list lengths are x and y , the merge takes $O(x+y)$ operations.

Crucial: postings sorted by docID.

Intersecting two postings lists (a “merge” algorithm)

INTERSECT(p_1, p_2)

```
1  answer  $\leftarrow \langle \rangle$ 
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$ 
3  do if  $\text{docID}(p_1) = \text{docID}(p_2)$ 
4      then ADD(answer,  $\text{docID}(p_1)$ )
5           $p_1 \leftarrow \text{next}(p_1)$ 
6           $p_2 \leftarrow \text{next}(p_2)$ 
7      else if  $\text{docID}(p_1) < \text{docID}(p_2)$ 
8          then  $p_1 \leftarrow \text{next}(p_1)$ 
9          else  $p_2 \leftarrow \text{next}(p_2)$ 
10 return answer
```

Boolean model

Boolean queries: Exact match

- The **Boolean retrieval model** is being able to ask a query that is a Boolean expression:
 - Boolean Queries are queries using *AND*, *OR* and *NOT* to join query terms
 - Views each document as a set of words
 - Is precise: document matches condition or not.
 - Perhaps the simplest model to build an IR system on
- Primary commercial retrieval tool for 3 decades.
- Many search systems you still use are Boolean:
 - Email, library catalog, macOS Spotlight



SEARCH

[Simple search](#)**Advanced search**[Full text search](#)

Insert search criteria

Search...

AND

Search...

AND

OR

NOT

Language:

- All languages -

Select criterions of search

Keywords

Keywords

+ Add row

Search

Reset

Search by catalogs:

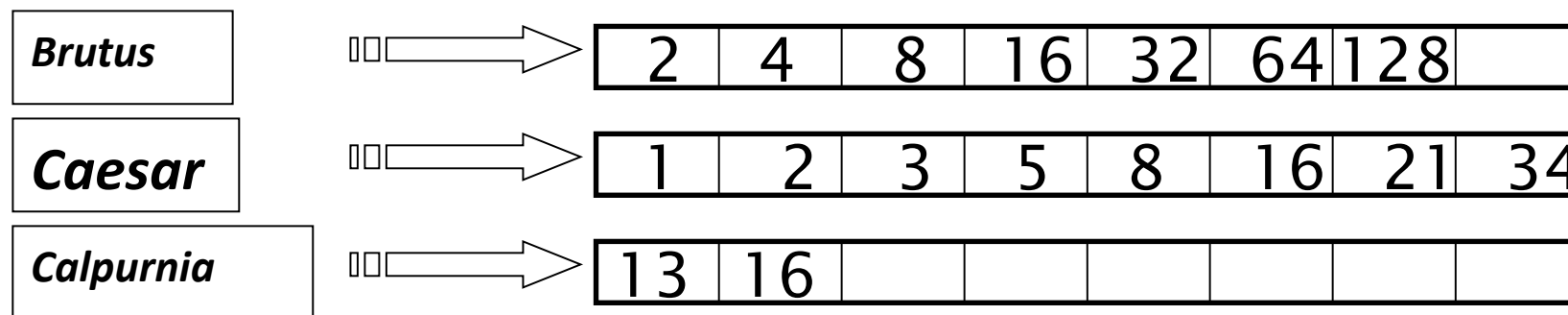
☐ All catalogs☐ Books☐ Maps☐ Media☐ Kit☐ e-Books☐ e-maps☐ Resources on Central
Asia☐ NU in History☐ Microforms☐ Donated books☐ Standards☐ Search in NU Communities

Example: WestLaw <http://www.westlaw.com/>

- Largest commercial (paying subscribers) legal search service (started 1975; ranking added 1992; new federated search added 2010)
- Tens of terabytes of data; ~700,000 users
- Majority of users *still* use boolean queries
- Example query:
 - What is the statute of limitations in cases involving the federal tort claims act?
 - **LIMIT! /3 STATUTE ACTION /S FEDERAL /2 TORT /3 CLAIM**
 - /3 = within 3 words, /S = in same sentence

Query optimization

- What is the best order for query processing?
- Consider a query that is an *AND* of n terms.
- For each of the n terms, get its postings, then *AND* them together.

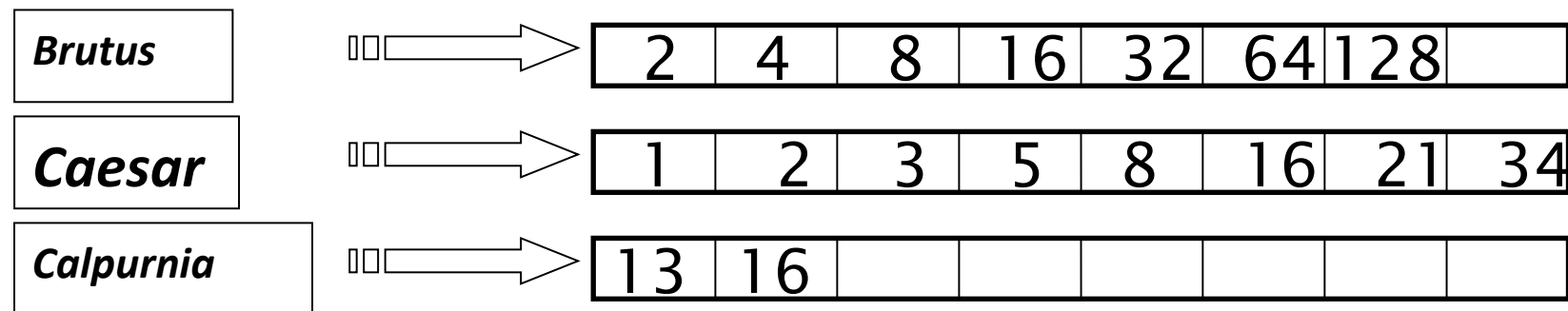


Query: **Brutus AND Calpurnia AND Caesar**

Query optimization example

- Process in order of increasing freq:
 - *start with smallest set, then keep cutting further.*

This is why we kept document freq. in dictionary



Execute the query as (***Calpurnia AND Brutus***) AND ***Caesar***.

Exercise

- Recommend a query processing order for

*(tangerine OR trees) AND
(marmalade OR skies) AND
(kaleidoscope OR eyes)*

- Which two terms should we process first?

Term	Freq
eyes	213312
kaleidoscope	87009
marmalade	107913
skies	271658
tangerine	46653
trees	316812

More general optimization

- e.g., (*madding OR crowd*) AND (*ignoble OR strife*)
- Get doc. freq.'s for all terms.
- Estimate the size of each *OR* by the sum of its doc. freq.'s (conservative).
- Process in increasing order of *OR* sizes.

Phrase queries and positional indexes

Phrase queries

- We want to be able to answer queries such as “***stanford university***” – as a phrase
- Thus, the sentence “*I went to university at Stanford*” is not a match.
 - The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works
 - Many more queries are *implicit phrase queries*
- For this, it no longer suffices to store only *<term : docs>* entries

A first attempt: Biword indexes

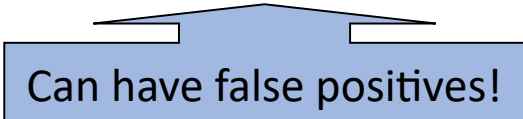
- Index every consecutive pair of terms in the text as a phrase
- For example, the text “Friends, Romans, Countrymen” would generate the biwords
 - *friends romans*
 - *romans countrymen*
- Each of these biwords is now a dictionary term
- Two-word phrase query-processing is now immediate.

Longer phrase queries

- Longer phrases can be processed by breaking them down
- ***stanford university palo alto*** can be broken into the Boolean query on biwords:

stanford university AND university palo AND palo alto

Without the docs, we cannot verify that the docs matching the above Boolean query do contain the phrase.



Can have false positives!

Issues for biword indexes

- False positives, as noted before
- Index blowup due to bigger dictionary
 - Infeasible for more than biwords, big even for them
- Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy

Solution 2: Positional indexes

- In the postings, store, for each ***term*** the position(s) in which tokens of it appear:

<***term***, number of docs containing ***term***;

doc1: position1, position2 ... ;

doc2: position1, position2 ... ;

etc.>

Positional index example

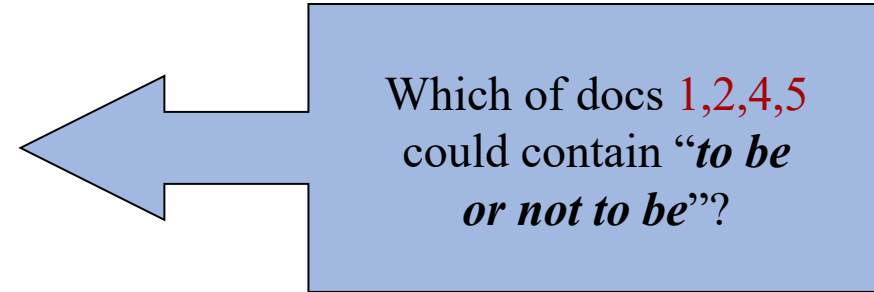
<*be*: 993427;

1: 7, 18, 33, 72, 86, 231;

2: 3, 149;

4: 17, 191, 291, 430, 434;

5: 363, 367, ...>



- For phrase queries, we use a merge algorithm recursively at the document level
- But we now need to deal with more than just equality

Processing a phrase query

- Extract inverted index entries for each distinct term: ***to, be, or, not.***
- Merge their *doc:position* lists to enumerate all positions with “***to be or not to be***”.
 - ***to:***
 - 2:1,17,74,222,551; 4:8,16,190,429,433; 7:13,23,191; ...
 - ***be:***
 - 1:17,19; 4:17,191,291,430,434; 5:14,19,101; ...
- Same general method for proximity searches

Proximity queries

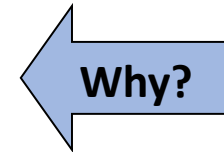
- LIMIT! /3 STATUTE /3 FEDERAL /2 TORT
 - Again, here, / k means “within k words of”.
- Clearly, positional indexes can be used for such queries; biword indexes cannot.
- Exercise: Adapt the linear merge of postings to handle proximity queries. Can you make it work for any value of k ?
 - This is a little tricky to do correctly and efficiently
 - See Figure 2.12 of *IIR*

Positional index size

- A positional index expands postings storage *substantially*
 - Even though indices can be compressed
- Nevertheless, a positional index is now standardly used because of the power and usefulness of phrase and proximity queries ... whether used explicitly or implicitly in a ranking retrieval system.

Positional index size

- Need an entry for each occurrence, not just once per document
- Index size depends on average document size
 - Average web page has <1000 terms
 - SEC filings, books, even some epic poems ... easily 100,000 terms
- Consider a term with frequency 0.1%



Document size	Postings	Positional postings
1000	1	1
100,000	1	100

Rules of thumb

- A positional index is 2–4 as large as a non-positional index
- Positional index size 35–50% of volume of original text
 - Caveat: all of this holds for “English-like” languages

Combination schemes

- These two approaches can be profitably combined
 - For particular phrases (***“Michael Jackson”***, ***“Britney Spears”***) it is inefficient to keep on merging positional postings lists
 - Even more so for phrases like ***“The Who”***
- Williams et al. (2004) evaluate a more sophisticated mixed indexing scheme
 - A typical web query mixture was executed in $\frac{1}{4}$ of the time of using just a positional index
 - It required 26% more space than having a positional index alone

Index compression

Index parameters vs. what we index

(details IIR Table 5.1, p.80)

size of	word types (terms)			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %	Size (K)	$\Delta\%$	cumul %
Unfiltered	484			109,971			197,879		
No numbers	474	-2	-2	100,680	-8	-8	179,158	-9	-9
Case folding	392	-17	-19	96,969	-3	-12	179,158	0	-9
30 stopwords	391	-0	-19	83,390	-14	-24	121,858	-31	-38
150 stopwords	391	-0	-19	67,002	-30	-39	94,517	-47	-52
stemming	322	-17	-33	63,812	-4	-42	94,517	0	-52

Exercise: give intuitions for all the '0' entries. Why do some zero entries correspond to big deltas in other columns?

Vocabulary size vs. collection size

- Heaps' law: $M = kT^b$
- M is the size of the vocabulary, T is the number of tokens in the collection
- Typical values: $30 \leq k \leq 100$ and $b \approx 0.5$
- In a log-log plot of vocabulary size M vs. T , Heaps' law predicts a line with slope about $\frac{1}{2}$
 - It is the simplest possible (linear) relationship between the two in log-log space
 - $\log M = \log k + b \log T$
 - An empirical finding (“empirical law”)

Heaps' Law

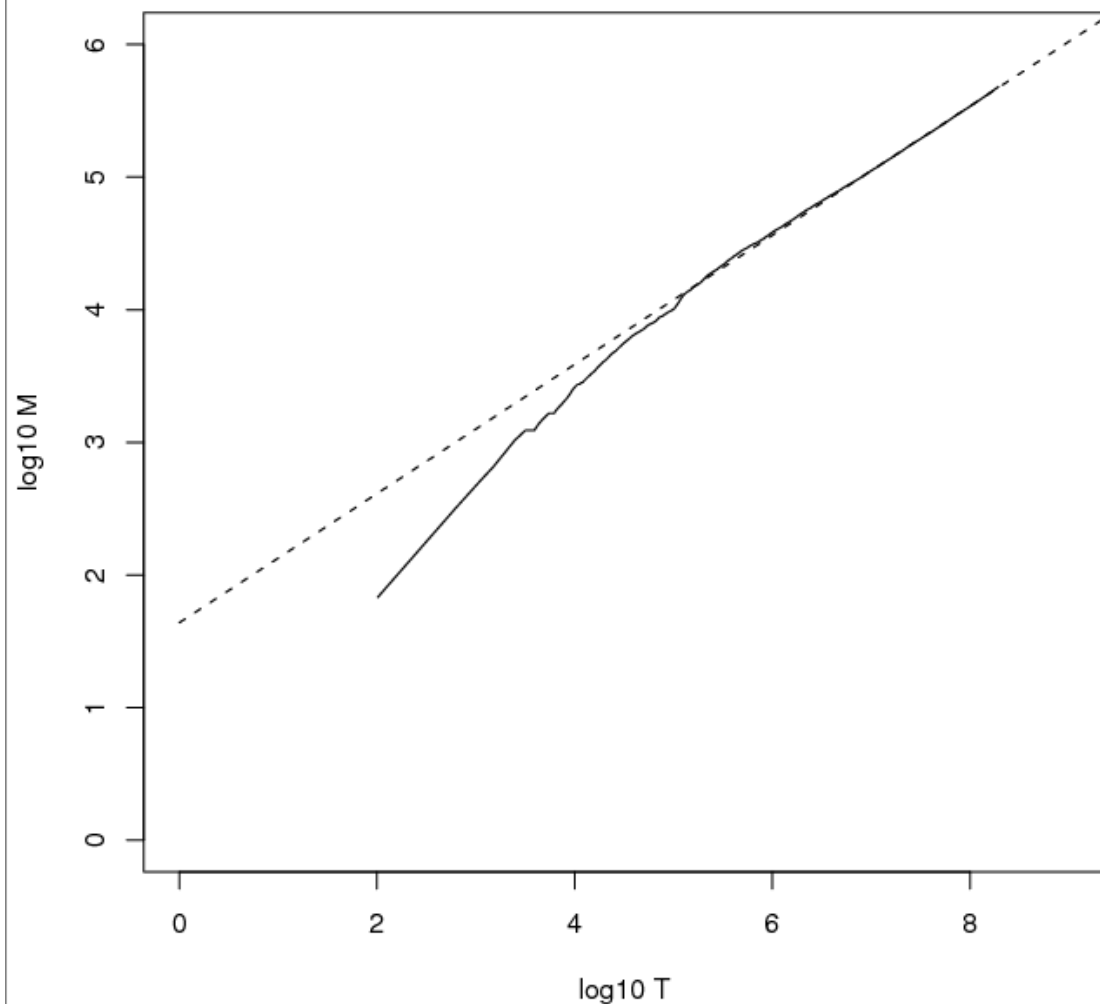
For RCV1, the dashed line

$\log_{10} M = 0.49 \log_{10} T + 1.64$ is the best least squares fit.

Thus, $M = 10^{1.64} T^{0.49}$ so $k = 10^{1.64} \approx 44$ and $b = 0.49$.

Good empirical fit for Reuters RCV1 !

For first 1,000,020 tokens,
law predicts 38,323 terms;
actually, 38,365 terms



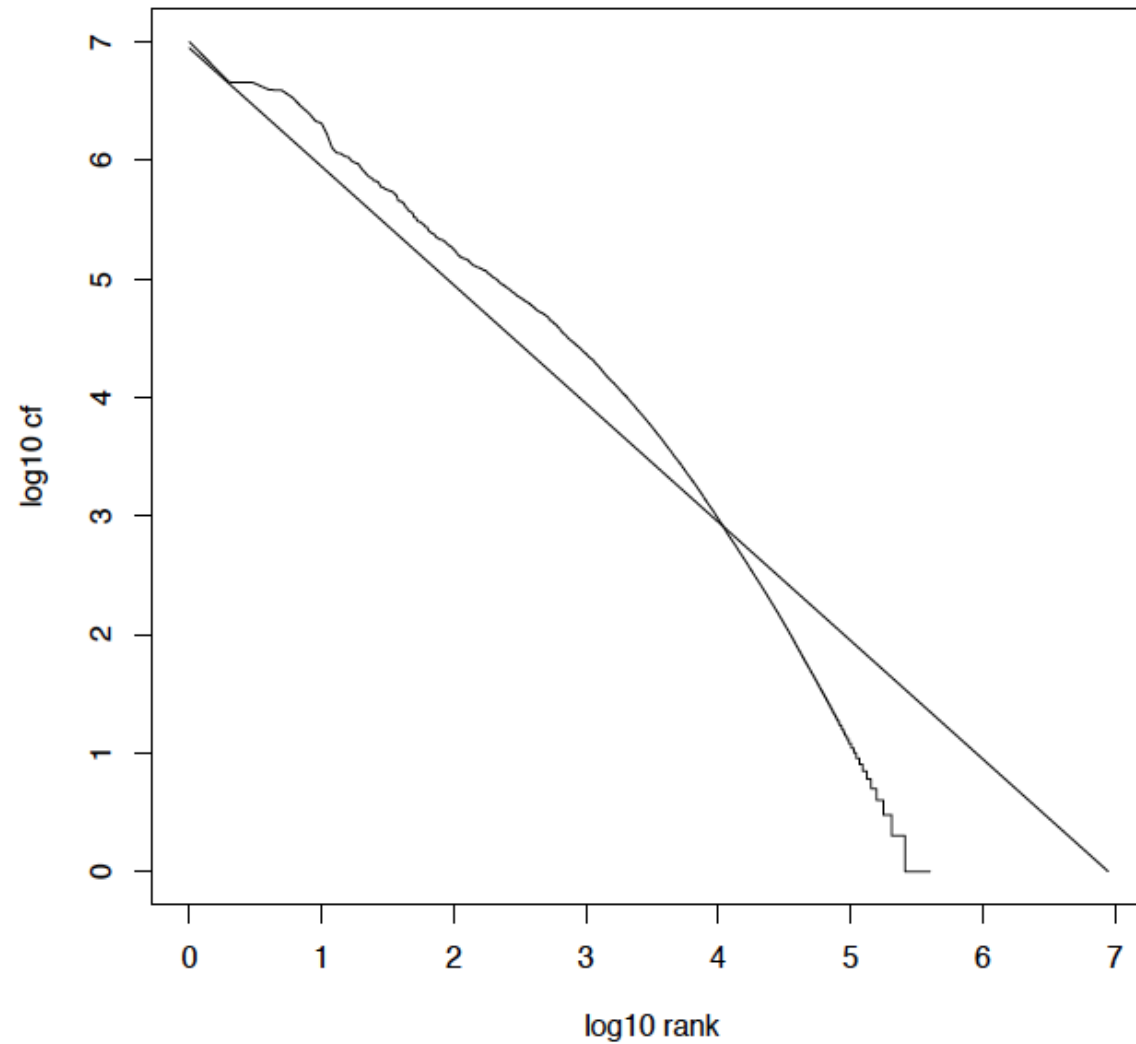
Zipf's law

- Heaps' law gives the vocabulary size in collections.
- We also study the relative frequencies of terms.
- In natural language, there are a few very frequent terms and very many very rare terms.
- Zipf's law: The i^{th} most frequent term has frequency proportional to $1/i$.
- $cf_i \propto 1/i = K/i$ where K is a normalizing constant
- cf_i is collection frequency: the number of occurrences of the term t_i in the collection.

Zipf consequences

- If the most frequent term (*the*) occurs cf_1 times
 - then the second most frequent term (*of*) occurs $cf_1/2$ times
 - the third most frequent term (*and*) occurs $cf_1/3$ times ...
- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
 - $\log cf_i = \log K - \log i$
 - Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship

Zipf's law for Reuters RCV1



Postings compression

- The postings file is much larger than the dictionary, factor of at least 10, often over 100 times larger
- Key desideratum: store each posting compactly.
- A posting for our purposes is a docID.
- For Reuters (800,000 documents), we would use 32 bits per docID when using 4-byte integers.
- Alternatively, we can use $\log_2 800,000 \approx 20$ bits per docID.
- Our goal: use far fewer than 20 bits per docID.

Postings: two conflicting forces

- A term like ***arachnocentric*** occurs in maybe one doc out of a million – we would like to store this posting using $\log_2 1M \approx 20$ bits.
- A term like ***the*** occurs in virtually every doc, so 20 bits/posting $\approx 2MB$ is too expensive.
 - Prefer 0/1 bitmap vector in this case ($\approx 100K$)

Gap encoding of postings file entries

- We store the list of docs containing a term in increasing order of docID.
 - **computer**: 33,47,154,159,202 ...
- Consequence: it suffices to store *gaps*.
 - 33,14,107,5,43 ...
- Hope: most gaps can be encoded/stored with far fewer than 20 bits.
 - Especially for common words

Three postings entries

	encoding	postings list					
THE	docIDs	...	283042	283043	283044	283045	...
	gaps		1	1	1		...
COMPUTER	docIDs	...	283047	283154	283159	283202	...
	gaps		107	5	43		...
ARACHNOCENTRIC	docIDs	252000	500100				
	gaps	252000	248100				

Variable length encoding

- Aim:
 - For *arachnocentric*, we will use ~ 20 bits/gap entry.
 - For *the*, we will use ~ 1 bit/gap entry.
- If the average gap for a term is G , we want to use $\sim \log_2 G$ bits/gap entry.
- Key challenge: encode every integer (gap) with about as few bits as needed for that integer.
- This requires a *variable length encoding*
- Variable length codes achieve this by using short codes for small numbers

Variable Byte (VB) codes

- For a gap value G , we want to use close to the fewest bytes needed to hold $\log_2 G$ bits
- Begin with one byte to store G and dedicate 1 bit in it to be a continuation bit c
- If $G \leq 127$, binary-encode it in the 7 available bits and set $c = 1$
- Else encode G 's lower-order 7 bits and then use additional bytes to encode the higher order bits using the same algorithm
- At the end set the continuation bit of the last byte to 1 ($c = 1$) – and for the other bytes $c = 0$.

Example

docIDs	824	829	215406
gaps		5	214577
VB code	00000110 10111000	10000101	00001101 00001100 10110001

Postings stored as the byte concatenation

000001101011100010000101000011010000110010110001

Key property: VB-encoded postings are uniquely prefix-decodable.

For a small gap (5), VB uses a whole byte.

The Importance of Interaction in Information Retrieval

- A keynote by Bruce Croft @ SIGIR2019

